

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

Zexin Fu, Diyou Shen

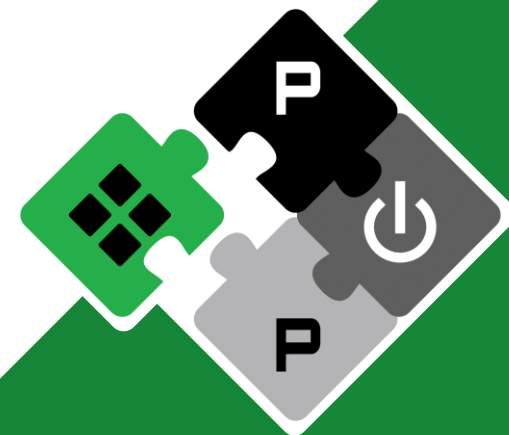
zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli
Luca Benini

avanelli@iis.ee.ethz.ch
lbenini@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!



@pulp_platform



pulp-platform.org



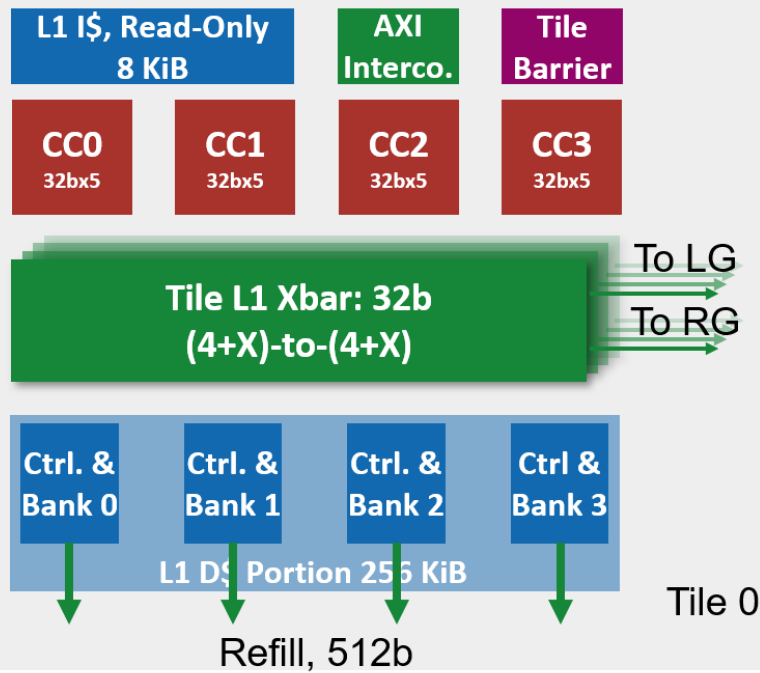
youtube.com/pulp_platform



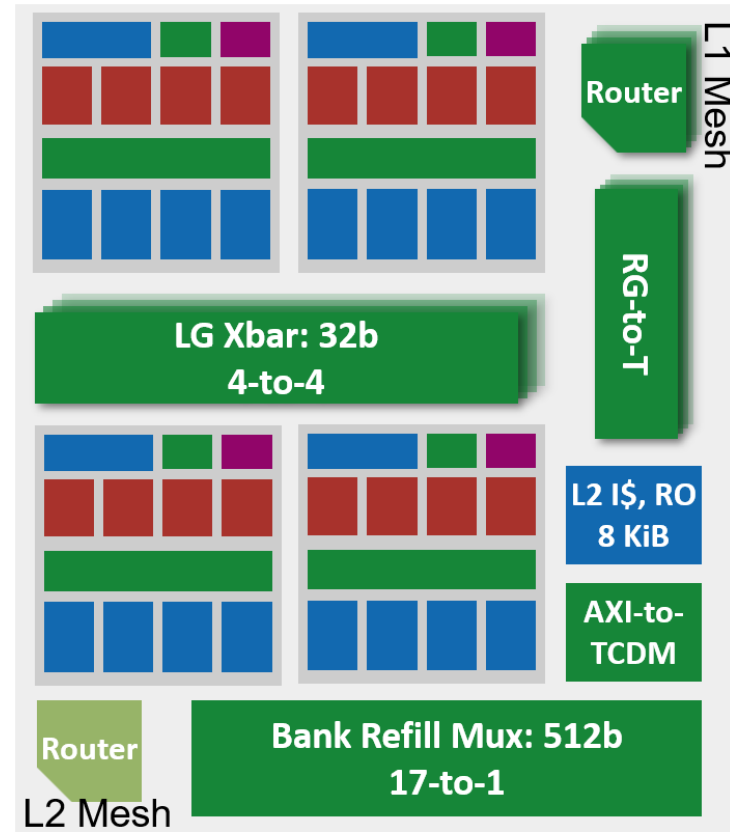
Hardware Development (Cluster)



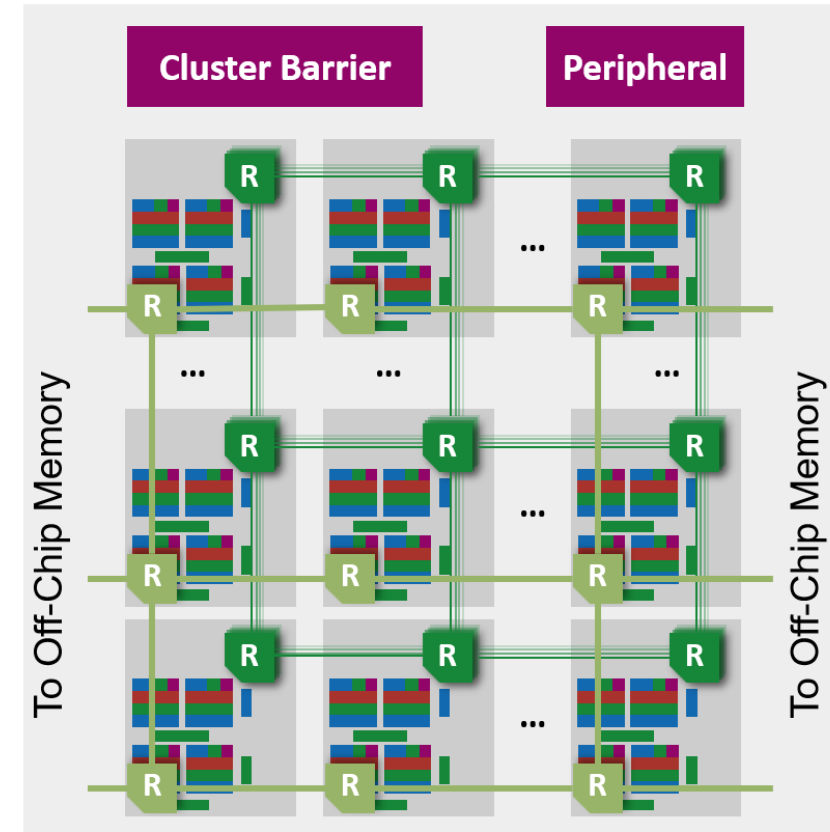
Tile



Group



Cluster



Hardware Development (Cluster)



- **Plans and Status**

- Benchmarking and performance tuning
- Some nice to have features
 - **DSE support for folded cache (Freezon)**
Freeze for now: more bugs than expected
 - **Group-level partitioning (Final Cleanups)**
Add by-group partitioning support. Switchable in software runtime.
One CI test fails, is debugging now
 - **More L2 channel support (Final Cleanups)**
Support both 8 and 16 channels for off-chip connection. (TeraPool uses 16 channels)
 - **Better dual-scalar core division (Idea)**
 - **Vector non-linear support (Planned)**
Merge a non-linear support from our colleagues, since it is needed for some log calculation



Hardware Development



- **By-Group Partitioning**

- Support all current configuration scheme
- Extra configuration to select whether to use tile-private or group-private.
- Do NOT support tile-private + group-share now. Plan to add it after more important tasks completes

- **Testing**

- Tested currently with fmatmul-1024x128x128 using all 256 spatz
- FPU utilization on hot cache: 92%
- FPU utilization on cold cache: 50-60% depending on dram fresh/interleaving

Private banks	Shared banks	Configuration
0	4	Fully shared (default)
1	3	Mixed
2	2	Mixed
3	1	Mixed
4	0	Fully private



Report



- **Status**

- Introduction (>90%)
 - Modified from annual report. May need some final tuning later
- Architecture (>85%)
 - Most parts completed
 - Missing some details for folded cache
- Physical Design (30%)
 - Waiting for results from new runs
- GVSoC (50%)
 - Basic structure completes. Need to add more details
- Runtime (>95%)
 - In general completes. May need some final cleanups.



Report



- **Status**

- Kernel Characterization (~30%)
 - Based on annual report version
 - Needs to adjust some parts after non-linear support integrated
- RLC Implementation (~30%)
 - Based on annual report version
 - Need some major updates since kernel has been updated
- Evaluation (0%)
 - Under data collection now
- Appendix (50%)
 - Added appendix for WT exploration, runtime functions and Spatz supported insn
 - Need NoC topology exploration parts.



Next Step

- **Diyou Side**

- Merge non-linear support and finish new PD runs
- Report writing

- **Zexin Side**

- GVSoC development, fix the dramsys under the cachepool_v3 target
- SW development based on the benchmark plan
- Finish the cts and routing of the group backend implementation



Timeline



		2025					2026							
		11	12	1	2	3	4	5	6	7	8	9	10	11
WP1 ManyCore Architecture	Multi-Tile Scaling [RTL] Complete													
	4/16-Group Scaling [GVSoC + RTL] Complete													
	Interco. Optimization (multi-group) [RTL] Almost Complete													
	Timing optimization for NoC [RTL] Ongoing by colleague													
	NoC Topology Exploration [RTL, GVSoC] Ongoing by student													
WP2 Cache	Cache partitioning [RTL] Complete													
	WT-L1 by master student [RTL] Complete													
	Folded Cache [GVSoC + RTL] Complete													



Timeline



		2025						2026						
		11	12	1	2	3	4	5	6	7	8	9	10	11
WP3 VPE	Shared vector core [RTL + GVSoc] Almost Complete													
WP4 Benchmark	Single-user kernel optimization [SW] Ongoing													
	Multi-user and more scenarios [SW] Ongoing													
Report	Final Report and Benchmark													



L1 Access Latency



- **Problem: increased L1 access latency when scaling up**
- **Solutions**
 - **(Complete, RTL & Physical) Hardware optimization**
Further optimize the interconnection and cache controller to reduce access latency on existing architecture
=> Retiming the current interconnection, remove unnecessary cuts in interco and cache
=> Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
Currently has reduced to 6 cycles:
Core => Xbar => AMO => Ctrl (2) => AMO => Core
 - **(Complete, RTL) Cache partitioning**
Partition the shared L1 to keep data localized to avoid remote access latency
 - **(Complete with some problems, RTL & Physical) WT-L1**
Explore and evaluate a private WT L1 for each core
=> Performance degrade on single tile
=> Needs more work to support larger system



L1 Access Latency



- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
 - **(Ongoing, Physical) Reduce hop latency**
Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
=> Reduce the latency by 1/2
 - **(Planned, Physical) Cut clk-domain (optional)**
If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
=> Effectively cut the latency by 1/3
 - **(Planned, GVSoc & Physical) NoC Topology**
Explore different NoC topologies
=> Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2



PE-Related



- **Problem: Improve scalar core performance**
- **Solutions**
 - **(Complete, RTL & GVSoc) Shared vector core**
Explore two Snitch sharing one Spatz configuration to improve scalar performance
Software complexity needs to be considered, especially on the possible conflicting use of register file.
Discussed Solution: critical section



RLC Workload



- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
 - **(Ongoing, SW) Kernel optimization**
Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
 - **(Planned, SW) Kernel development**
Implement the missing use cases for multi-user



Thank you!

Q&A

